



ITI · AI AGENTS DEVELOPMENT USING PYTHON TRACK

Python Framework with FastAPI

Eng. Fatma Elraey

Lecture 3 —SQLAlchemy ORM with SQLite

Field	Detail
Instructor	Eng. Fatma Elraey
Course	Python Framework with FastAPI
Session	Lecture 3 of 5
Focus	SQLite database file, a simple SQLAlchemy ORM model, direct Session(engine), and basic CRUD persistence.

Welcome!

This session continues the same Documents API. We now move data out of temporary Python memory into a small SQLite database file. The goal is to understand ORM persistence with the fewest moving parts: one engine, one model, one session block, and basic CRUD.

Today's Outline

1. Why persistence is needed and what ORM means
2. Create a SQLite database file with SQLAlchemy
3. Define the Document ORM model
4. Open a simple Session with Session(engine)
5. Basic CRUD: create, read, update, delete

01

Why a Database? ORM Mental Model

In Lectures 1 and 2, data could live in a Python list or dictionary. That is easy to learn from, but it disappears when the process restarts. SQLite stores the same data in a real database file, and SQLAlchemy ORM lets us work with Python Document objects instead of writing SQL strings for every operation.

An ORM maps Python objects to database rows. Instead of building SQL strings for every operation, application code works with mapped classes while SQLAlchemy generates SQL and tracks session state.

So ORM stands for **Object-Relational Mapping**, lets explain

Relational database has:

- Tables
- Rows
- Columns

Python has:

- Classes
- Objects
- Attributes

ORM maps between these two, for example:
if we have a database row with data:

```
id      = 104
title   = "FastAPI Notes"
priority = 2
```

but in Python we deal with it as an object:

```
Document(
    id=104,
    title="FastAPI Notes",
    priority=2
)
```

So instead of writing

```
SELECT * FROM documents WHERE id = 104;
```

We deal with python classes and objects:

```
doc = db.get(Document, 104)
```

Simple ORM persistence: FastAPI → SQLAlchemy → SQLite

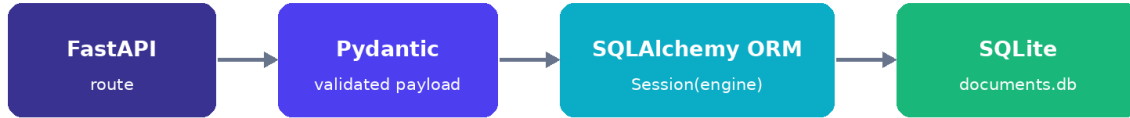


Figure 3.1 — FastAPI validates the request, SQLAlchemy maps Python objects, and SQLite stores the rows in documents.db.

```
# Instructor example – memory vs persistence
IN MEMORY documents = [...] → easy, but disappears after restart
SQLITE FILE documents.db → saved on disk
PYTHON OBJECT Document(id=104, title="FastAPI Notes")
DATABASE ROW id=104, title="FastAPI Notes"
```

PRO TIP For this lecture, think of ORM as a translator between Python objects and database rows. Keep the first example small.

02 SQLite + SQLAlchemy Setup

One engine + one Base class

First:

```
pip install sqlalchemy
```

```
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase

engine = create_engine(
    "sqlite:///documents.db"
)

class Base(DeclarativeBase):
    pass
```

- **Engine** — points SQLAlchemy at the database we want to use.
- **SQLite** — stores the database in one local file: documents.db.
- **Base** — parent class used by our ORM models.

03 Define the Document ORM Model

A Python class mapped to the documents table

```
from sqlalchemy import String, Text
from sqlalchemy.orm import Mapped, mapped_column

class Document(Base):
    __tablename__ = "documents"

    id: Mapped[int] = mapped_column(
        primary_key=True
    )

    title: Mapped[str] = mapped_column(
        String(120)
    )

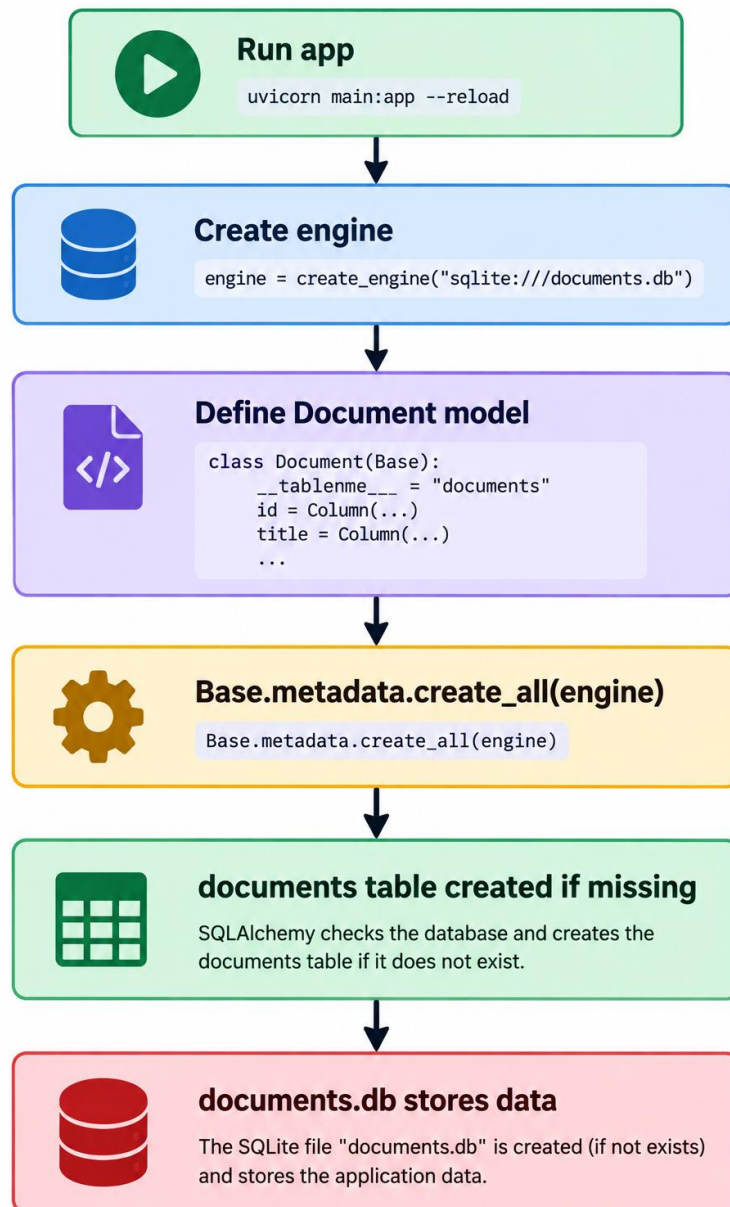
    content: Mapped[str] = mapped_column(
        Text
    )

    priority: Mapped[int] = mapped_column(
        default=1
    )

Base.metadata.create_all(engine)
```

The class maps to the documents table. Each attribute maps to a database column. For this beginner example, **create_all()** creates the table when it is missing.

Python ORM Object		Database Row
doc.id	↔	id
doc.title	↔	title
doc.content	↔	content
doc.priority	↔	priority



✓ QUICK CHECK

Q1. What does `__tablename__` define?

Answer: The database table name.

Q2. What does `Base.metadata.create_all(engine)` do?

Answer: It creates missing tables defined by the ORM models.

04 Use Session(engine) Directly

Simple session pattern

```
from sqlalchemy.orm import Session

@app.get("/documents")
def list_documents():
    with Session(engine) as db:
        return db.query(Document).all()
```

```
# Instructor example – the with block
OPEN with Session(engine) as db:
WORK db.query(), db.get(), db.add(), db.delete()
CLOSE leaving the with block closes the Session
SIMPLE VERSION no SessionLocal or Depends yet
```

PRO TIP *Keep one Session block inside each endpoint in this beginner version.*

05

CRUD with the SQLite File

Create + read

```

@app.post("/documents")
def create_document(payload: DocumentCreate):
    with Session(engine) as db:
        doc = Document(
            title=payload.title,
            content=payload.content,
            priority=payload.priority,
        )
        db.add(doc)
        db.commit()
        db.refresh(doc)
        return doc

@app.get("/documents/{document_id}")
def get_document(document_id: int):
    with Session(engine) as db:
        doc = db.get(Document, document_id)
        if doc is None:
            raise HTTPException(404, "Document not found")
        return doc

```

- **add()** — puts the new ORM object into the current Session.
- **commit()** — saves database changes.
- **refresh()** — reloads values generated by the database, such as id.
- **get()** — reads one row by its primary key.

Update + delete

```

@app.put("/documents/{document_id}")
def update_document(document_id: int, payload: DocumentCreate):
    with Session(engine) as db:
        doc = db.get(Document, document_id)
        if doc is None:
            raise HTTPException(404, "Document not found")
        doc.title = payload.title
        doc.content = payload.content
        doc.priority = payload.priority
        db.commit()
        db.refresh(doc)
        return doc

@app.delete("/documents/{document_id}")
def delete_document(document_id: int):
    with Session(engine) as db:
        doc = db.get(Document, document_id)
        if doc is None:
            raise HTTPException(404, "Document not found")

```

```
db.delete(doc)
db.commit()
return {"message": "Document deleted"}
```

✓ QUICK CHECK

Q1. Which call actually saves database changes?

Answer: `db.commit()`.

Q2. Why use `with Session(engine) as db`?

Answer: It gives the endpoint a database Session for the block and closes it afterward.

Work through these on your own before checking the marked answers.

Practice Question 1

What does an ORM map?

- A. HTML to CSS
- B. Python objects to database rows
- C. Files to images
- D. HTTP to SMTP

Practice Question 2

Where is the SQLite database stored in this lecture?

- A. In documents.db
- B. In a browser cookie
- C. In an HTML template
- D. In a Python list only

Practice Question 3

Which line creates missing ORM tables in this simple example?

- A. `db.refresh(doc)`
- B. `Base.metadata.create_all(engine)`
- C. `app = FastAPI()`
- D. `json.dump(...)`

Practice Question 4

What does `with Session(engine) as db` provide?

- A. A database Session for that block
- B. A new FastAPI app
- C. A file upload
- D. A Pydantic model

Practice Question 5

After changing or deleting a row, what saves the change?

- A. `db.get()`
- B. `db.commit()`
- C. `db.refresh()` only
- D. return doc

Answers

Practice Question 1: B ✓ — ORM maps Python objects to relational database rows.

Practice Question 2: A ✓ — SQLite data is stored in the documents.db file.

Practice Question 3: B ✓ — `create_all()` creates missing tables from the ORM metadata.

Practice Question 4: A ✓ — the `with` block provides a SQLAlchemy Session and closes it afterward.

Practice Question 5: B ✓ — `commit()` persists the update or delete in the database.